

INTUITION_OS_MASTER_SPEC.md

PART 4

CLAUDE BUILD INSTRUCTIONS

IMPLEMENTATION BLUEPRINT

Version 1.0

Purpose

This section exists specifically for Claude Code.

The goal is to remove ambiguity.

Claude should be able to build the entire MVP from this specification with minimal clarification.

Project Goal

Build a production-quality MVP of Intuition OS.

Primary objective:

Validate whether users engage with mentorship-driven learning instead of solution-driven learning.

Do not optimize for scale.

Do not optimize for complexity.

Optimize for learning velocity.

Build Philosophy

The biggest risk is not technical.

The biggest risk is product validation.

Therefore:

Build the smallest possible version capable of validating the mentor experience.

Implementation Priority

Order matters.

Build in this sequence.

Never reverse it.

Phase 1

Authentication

Goal:

User can create account.

Requirements:

- Google OAuth only
- JWT authentication
- Refresh tokens
- Persistent sessions

Do NOT implement:

- Email/password
 - Password reset
 - Username system
-

Phase 2

Learning Session System

Goal:

Track problem-solving sessions.

Requirements:

- Create session
- Update session
- Close session

Store:

- Problem title
 - URL
 - Difficulty
 - Tags
 - Start time
 - End time
-

Phase 3

Browser Extension

Goal:

Capture LeetCode activity.

Technology:

- Plasmio
- React
- TypeScript

Required Features:

Detect:

- Problem title
- Difficulty
- Tags
- URL

Create session automatically.

Phase 4

Mentor MVP

Goal:

User can request guidance.

Requirements:

Need Help button.

Diagnostic flow.

Escalation engine.

LLM integration.

Phase 5

Reflection Engine

Goal:

Generate learning summary.

Requirements:

Reflection generation.

Evidence extraction.

Storage.

Phase 6

Dashboard

Goal:

Display learning history.

Requirements:

- Session history
- Reflections
- Strengths
- Weaknesses

Repository Structure

Monorepo.

Root

``text id="1hvjr" intuition-os/

apps/

packages/

infrastructure/

docs/

```
---

# Apps

``text id="z2mzlb"
apps/

dashboard/

extension/

backend/
```

Dashboard

Next.js application.

Responsibilities:

- Authentication
 - Dashboard
 - Profile
 - Session History
-

Extension

Plasmo application.

Responsibilities:

- Session tracking
 - Mentor interface
 - Event collection
-

Backend

FastAPI application.

Responsibilities:

- APIs
 - AI orchestration
 - Database
-

Packages

Shared code.

```text id="w3kpyz" packages/

types/

prompts/

shared/

```

Infrastructure

```text id="4iq5eh"  
docker/  
  
terraform/ (future)
```

Documentation

```
```text id="ozdt9v" docs/
```

```
product/
```

```
architecture/
```

```
api/
```

```

Backend Folder Structure

```text id="wg47ae"  
backend/  
  
app/  
  
api/  
  
services/  
  
models/  
  
schemas/  
  
repositories/  
  
prompts/  
  
utils/
```

Service Design

Each domain gets:

Repository

Service

Schema

Routes

Example:

```
```text id="1o6zdx" session/
```

session\_repository.py

session\_service.py

session\_schema.py

session\_router.py

```

Frontend Design System

Style:

Clean

Professional

Minimal

Not playful.

Not gamified.

Visual Inspiration
```



Combination of:

- Linear
- Notion
- Vercel
- Stripe Dashboard

---

Avoid:

- Bright colors
- Excessive gradients
- Gaming aesthetics

---

# Dashboard Layout

Top:

Mentor Summary

Middle:

Strengths

Weaknesses

Bottom:

Session Timeline

---

# Mentor UI

The mentor should not resemble ChatGPT.

Avoid:

Large chat interface.

Instead:

Guided interaction.

---

Structure:

Current Session

↓

Need Help

↓

Diagnostic Questions

↓

Mentor Guidance

---

# Mentor Escalation Logic

State Machine.

---

Level 1

Question Only

---

Level 2

Directional Guidance

---

Level 3

Pattern Guidance

---

Level 4

Strong Hint

```

Level 5

Solution Path

Never skip levels automatically.

AI Provider Architecture

Must support provider abstraction.

Interface:

```python id="zff7q2"  
class LLMPProvider:  
  
    def generate(self, prompt):  
        pass
```

Providers:

Claude

OpenAI

Gemini

Switching providers should require configuration only.

No business logic changes.

Prompt Management

Store prompts separately.

Never hardcode prompts inside services.

Structure

```text id="vuxx4n" prompts/

mentor/

reflection/

evidence/

```

Mentor Prompt Rules

Must include:

- Current Problem
- Session Context
- User Response
- Historical Evidence

Must not include:

Raw database dumps.

Reflection Prompt Rules

Generate:

- Summary
- Learning Insight
- Evidence
- Recommendation

Never generate:

Personality assessments.

Evidence Extraction Rules
```

Valid:

Recovered From TLE

Needed Hint Level 2

Solved Without Editorial

---

Invalid:

Smart

Creative

Lazy

Talented

Gifted

---

# API Standards

REST.

JSON.

Versioned.

Example:

```
```text id="s0oqxz"  
/api/v1
```

Response Format

Success:

```
```json id="e6vfls" { "success": true, "data": {} }
```

```

```

Failure:

```
```json id="0gk3c6"
{
  "success": false,
  "error": {}
}
```

Error Handling

Use centralized exception middleware.

No raw exceptions.

Database Standards

Use:

UUID primary keys.

Timestamps everywhere.

Soft deletes only if necessary.

Migrations

Alembic.

Every schema change must be migration driven.

Caching

Redis.

Use only when necessary.

Do not prematurely optimize.

Logging

Structured logging.

Include:

- User ID
- Session ID
- Service

Never log:

Tokens

Sensitive data

Analytics

Track:

- Mentor requests
- Hint levels
- Reflection generation
- Session completion

Do not build analytics dashboard initially.

Testing Requirements

Backend:

pytest

Frontend:

Vitest

Extension:

Playwright

Deployment

MVP:

Frontend:

Vercel

Backend:

Railway or Render

Database:

Neon PostgreSQL

Redis:

Upstash

Cost Optimization

Do not call LLM unnecessarily.

Mentor requests only.

Reflection generation only.

No background AI processing.

Technical Debt Rules

Allowed:

Simple solutions.

Not Allowed:

Architectural shortcuts that break future mentor memory.

Features Explicitly Deferred

Do not build:

- Code execution
 - Contest support
 - Gamification
 - Social feed
 - Friend system
 - Team solving
 - Rating prediction
 - Learning graph visualization
 - Mobile app
-

MVP Definition

The MVP is complete when:

A user can:

Install extension

↓

Login

↓

Open LeetCode problem

↓

Session starts



Request mentor help



Receive guidance



Solve problem



Receive reflection



View history

Nothing else is required.

Claude Final Instruction

When implementation decisions are unclear:

Choose simplicity.

When architecture decisions are unclear:

Optimize for maintainability.

When feature decisions are unclear:

Favor learning outcomes over engagement mechanics.

The purpose of Intuition OS is not to maximize activity.

The purpose of Intuition OS is to improve independent problem-solving ability.

END OF PART 4